

Algorithm Analysis

©2020 CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

- Explain the value of algorithm analysis.

- Describe the use of abstraction to quantify time and space in analysis.

- Explain the value and purpose of best-case, worst-case, and expected-case analyses.

- Employ best- and worst-case analyses for time and space for algorithms involving loops, conditionals and function calls.

- Explain the relationships between basic polynomial complexity classes.

- Explain what is meant by "best" and "worst" case behavior of an algorithm. [Partial, AL-BA-1]

- Determine informally the time and space complexity of simple algorithms. [Usage, AL-BA-3]

Four Abstractions

Cost of Computation

Programs use **time, memory, other valuable resources**

Resources are not free! They have costs.

We need a way to assess these costs scientifically (quantitatively).

Algorithm vs Program

Algorithm

- A sequence of actions that describe how to perform a task or solve a problem.

- Makes **no commitment** to a programming language.

Program

- A concrete realization of an algorithm, expressed in a programming language.

- An algorithm can be realized by many different programs.

When we assess the costs of a program, we ignore the variable aspects, and assess the underlying algorithm.

We will use **abstraction** to ignore variable aspects.

Abstraction #1: Time

Consider two scenarios:

Program A executed on a computer built in 2017.

Program A executed on a computer built in 1967.

Question: which program runs faster?

Answer: We have to separate the costs of the program from the abilities of the computer.

Abstraction:

Measure time in terms of computational steps.

Program A requires a number of steps.

Different computers do the steps faster or slower.

What is a computational step?

Any of the following is a computational step:

An arithmetic operation: +, *, -, /, etc

A logical operation: and, not, or

Numeric relational operations: <, >=, etc

Reference equality: is

Assigning a value to a variable

Accessing an element in a Python list (or numpy array)

Calling a function, returning a value

Cost of expressions used as operands and function bodies must be assessed separately

We will review this idea later.

How many computational steps?

(a) Example 1:

```
x = 3*y+4
```

Solution

Answer: 3 steps

(b) Example 2:

```
x = data[2]*data[3]
```

Solution

Answer: 4 steps

(c) Example 3:

```
x = 3*math.sqrt(data[i+1])
```

Solution

Answer: 5 steps, plus whatever sqrt is doing.

How many computational steps?

(a) Example 4:

```
1 total = 0
2 for i in range(10):
3     total = total + 1
```

Solution

Line 1: 1 step

Line 3, once: 2 steps

Line 3, 10 times: 20 steps

Line 2: 10 assignments, 9 additions, 19 steps

Total: $1 + 20 + 19 = 40$ steps

(b) Example 5:

```
1 total = 0
2 for i in range(N):
3     total = total + 1
```

Solution

Line 1: 1 step

Line 3, once: 2 steps

Line 3, N times: $2N$ steps

Line 2: N assignments, $N - 1$ additions, $2N - 1$ steps

Total: $1 + 2N + 2N - 1 = 4N$ steps

(c) Example 6:

```
1 total = 0
2 for i in range(N):
3     for j in range(N):
4         total = total + 1
```

Solution

Line 1: 1 step

Line 4, once: 2 steps

Line 4, N times: $2N$ steps

Line 3: N assignments, $N - 1$ additions, $2N - 1$ steps

Lines 3-4, once: $4N - 1$ steps

Lines 3-4, N times: $4N^2 - N$ steps

Line 2: N assignments, $N - 1$ additions, $2N - 1$ steps

Total: $1 + 2N - 1 + 4N^2 - N = 4N^2 + N$ steps

Abstraction #2: Input size

Consider two scenarios:

Program A executed on a list of size 10.

Program A executed on a list of size 1000.

Question: which program runs faster?

Answer: We have to express the cost as a function of the input size.

Abstraction:

Parameterize the size of the problem.

Express the number of steps taken as a function of the problem size.

Program A may take more steps on larger lists.

Parameterization

Parameterization assigns a mathematical name to a quantity that could change, or is unknown.

e.g. A list of size N , a box of size $L \cdot W \cdot H$, etc.

Express the size of a problem using parameters

Usually expressed as N , M , etc.

Express the number of steps required by an algorithm in terms of the problem size

e.g. Algorithm A takes $3N + 12$ steps to complete for input of size N

Parameterization examples (simple)

What are the input size parameters for the following examples?

- (a) Function to find the maximum value in a list (1D).

Solution

The length of the list.

- (b) Function to calculate the average value of a list (1D).

Solution

The length of the list.

- (c) Function to display all permutations of a list of items.

Solution

The length of the list.

Parameterization examples (more simple)

What are the input size parameters for the following examples?

- (a) Function to check if a Latin Square is valid.

Solution

The size of one side of the square.

- (b) Function to evaluate an expression, if:

The symbols are already in a list, e.g.: ['(', 3, '+', 1, ')']

Solution

The length of the list.

The expression is a string, e.g.: '(3 + 1)'

Solution

The length of the string.

- (c) Function to enqueue a value to a given queue.

Solution

Size of the queue.

- (d) Function to remove a given value from a node-based Linked List.

Solution

The length of the chain.

Parameterization examples

The Sieve of Eratosthenes

Uses a list of **boolean** values

One entry per positive integer from 2 to n

Value at index i reflects whether i is prime or not

Initially, all **True**

For every number i from $2 \dots n$,

If i is prime, mark its multiples as not prime

e.g. 2 is a prime number, so mark 4, 6, 8, etc. as not prime

What are the parameter(s) that indicate problem size?

Solution

The value n .

Parameterization examples (trickier)

The Gambler's Ruin Problem

50%-50% chance to win

Winning **earns** 1 dollar, losing **removes** 1 dollar

Gambler has a **stake** and a **goal**

Gambler plays games until **success** or **failure**.

To estimate the probability, we simulate the game a number of times.

What are the parameters that indicate problem size?

Solution

The number of trials. The distance between the stake and the goal, and the stake and 0.

Abstraction #3: Categories

Consider two scenarios:

Program A requires $3N + 12$ steps.

Program B requires $3N + 11$ steps.

(Both programs solve the same problem.)

Question: which program runs **faster**?

Answer: We ignore differences that don't matter.

Abstraction:

Express the cost of a program by placing it in a category.

All programs in the category are considered **equally efficient**.

Program A and **Program B** are in the same category.

How much difference makes a difference?

Suppose **Program A** requires $3N + 12$ steps.

For very large N , the extra 12 steps is **negligible**.

Our categories **ignore negligible details**.

For very large N , costs are (nearly) proportional to N .

Our categories **ignore constants of proportionality**.

Asymptotic Analysis: Large problem sizes

When we consider very large problem sizes, we ignore

Negligible costs

Constants of proportionality

This technique is called **Asymptotic analysis**

We use **Big-O Notation** to indicate the results of asymptotic analysis.

Suppose **Program A** requires $3N + 12$ steps.

We say **Program A** asymptotically requires $O(N)$ steps.

Big-O Notation

Big-O is a formal notation for expressing an algorithm's asymptotic behaviour

The O indicates asymptotic analysis.

e.g. $O(N)$, $O(N^3)$, $O(N^k)$ etc.

The N is a problem size parameter, for a given algorithm.

If we say that an algorithm requires $O(N^2)$ steps, we mean that as N increases, the number of steps needed increases no more quickly than N^2 .

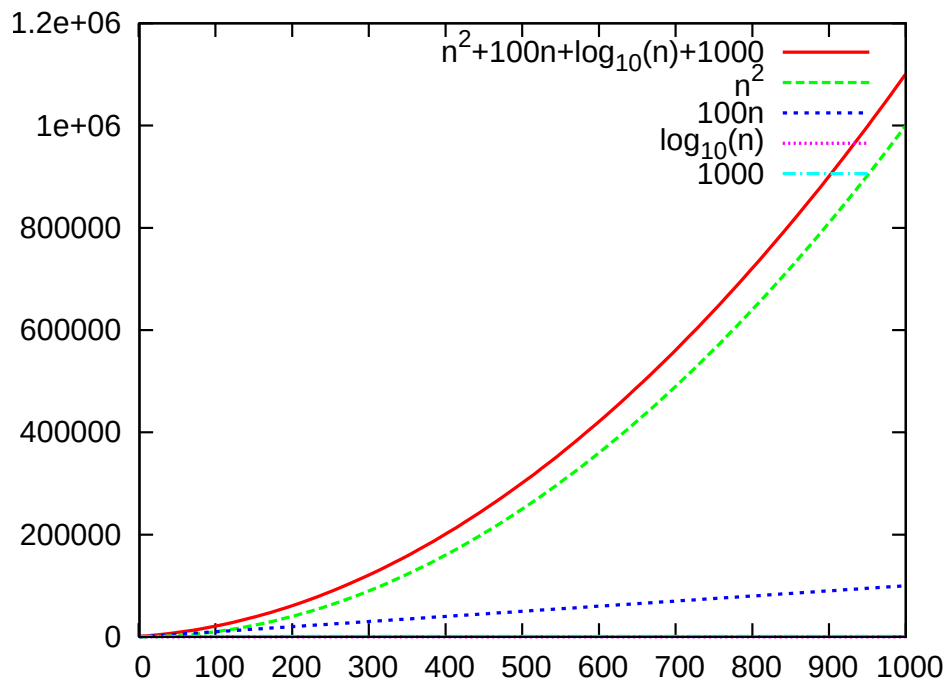
Big-O Categories

$$\begin{aligned} O(1) &< O(\log_2 N) < O(N) < O(N \log_2 N) \\ &< O(N^2) < O(N^3) < \dots < O(N^k) \\ &< O(2^N) < O(N!) < O(N^N) \end{aligned}$$

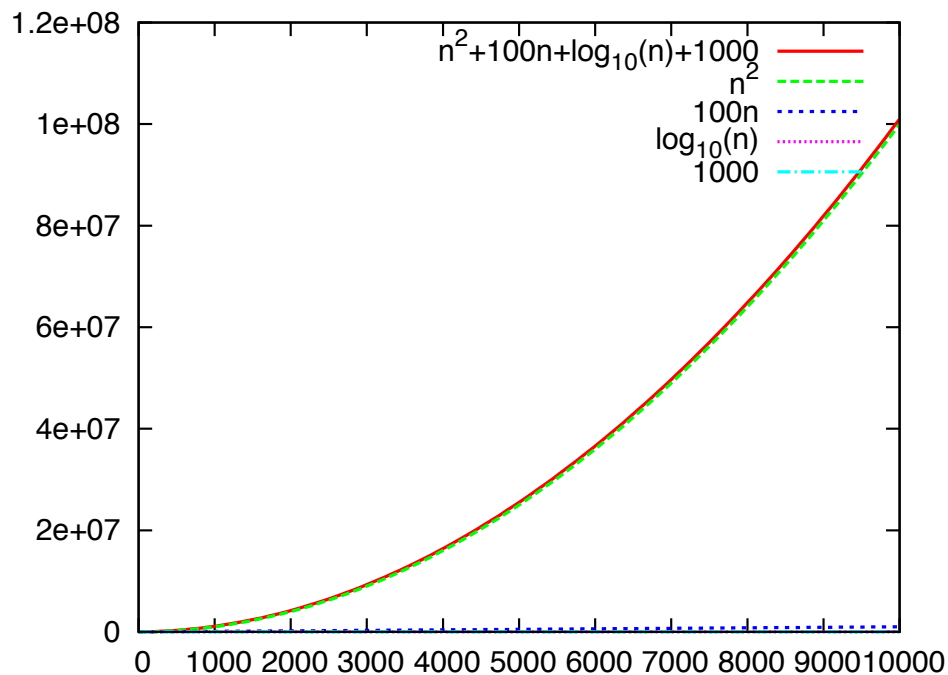
Here, $<$ means increases more slowly than, in the context of increasing N .

The listing here is not complete. For example, $O(N^2 \log N)$ could be included.

Big-O categories, visually



Big-O categories, bigger values of n



What Big-O Categories really mean

Informally, $O(N^2)$ means that computational costs increase **no more quickly than** N^2 .

But N **also increases no more quickly than** N^2

A Big-O category suggests an **upper limit** on the way costs increase with N

When we categorize an algorithm or program, we want the upper limit to be

As far "left" in the $<$ order as possible.

As "low" as possible in a graph.

Abstraction #4: Worst-case behaviour

Consider linear search for a given value in a given list.

The value appears as the **first** value in the list.

The value appears as the **last** value in the list.

The value **does not appear** in the list.

Question: Which of the cases is **representative** of the cost of a linear search?

Answer: We use the worst-case, pessimistically.

Abstraction:

The worst-case behaviour is used to categorize an algorithm.

Clarifying worst-case behaviour

The worst-case for an algorithm **forces** it to do the **maximum** amount of work **for a given problem size**.

It is always wrong to mention problem size when discussing best and worst case behaviour.

1. The best case is **not** when a problem is **small**.
2. The worst case is **not** when a problem is **big**.

Many algorithms show no difference between best and worst cases.

We would prefer to use average-case in principle, but except for really trivial algorithms, average-case analysis is usually too hard.

Examples: Worst-case behaviour

In terms of the time costs, what are the best and worst cases for the following?

- (a) Linear search in a list.

Solution

Best case: When the item you're looking for is first

Worst case: When the item you're looking for is last

- (b) Binary search in a list.

Solution

Best case: When the item you're looking for is the first thing you look at, i.e., right in the middle of the whole list

Worst case: When the item you're looking for is the last thing you can look at, after you've narrowed down the range to a single value.

- (c) Calculating the average of a list of values.

Solution

Best and worst cases are the same. You have to add everything no matter what.

- (d) Inserting a value into a node-based chain of nodes.

Solution

Best case: When you add the item first.

Worst case: When you have to walk all the way to the end of the list.

- (e) Gambler's ruin.

Solution

Best case: When you're lucky, and you always win to reach the goal (or always lose, to reach failure)

Worst case: When you alternate winning 1 and losing 1. There are other cases equally expensive: winning/losing 2 alternately, etc.

- (f) Sieve of Eratosthenes.

Solution

Best case and worst case are the same: you have to consider $n - 1$ integers.

Exercise 2

Categorize the following program in terms of Big-O, by counting steps. What is the Big-O analysis for the best and worst case?

```

1 def roll_dice():
2     return random.randint(1,6) + random.randint(1,6)
3
4 def rolling(n_rolls):
5     low_rolls = 0      # number rolls whose sum is <= 4
6     high_rolls = 0     # number rolls whose sum is >= 10
7     dice_rolls = [0]*n_rolls
8     for roll in range(n_rolls):
9         dice_rolls[roll] = roll_dice()
10        if dice_rolls[roll] <= 4:
11            low_rolls += 1
12        elif dice_rolls[roll] >= 10:
13            high_rolls += 1
14    print(low_rolls, high_rolls)
15    print(dice_rolls)

```

Solution

You could try to count the steps, but there's an easier way. We look at this problem again later!

Asymptotic Analysis

Asymptotic Analysis: A simplified algebra

Before we perfect the counting of steps, we introduce a few rules about Big-O notation.

This will make counting steps so much easier.

None of this is hard, but it does look like math.

We start with rules and simplifications.

Practical examples will come after.

Asymptotic Analysis Principles

Identities

Let A , B be any mathematical expression. In asymptotic analysis, the following identities hold:

$$O(A) + O(B) = O(A + B)$$

$$O(A) \times O(B) = O(A \times B)$$

$$A \times O(B) = O(A \times B)$$

Asymptotic Analysis Principles

Replacing constants with 1

In asymptotic analysis:

If A is a constant, $O(A) = O(1)$.

If A is a constant, and B is any expression, $O(A \times B) = O(B)$.

Drop lower order terms

In asymptotic analysis, if $O(A) < O(B)$, then $O(B) + O(A) = O(B)$.

Example

Use the rules for Big-O to simplify this expression:

$$\begin{aligned} &O(3N + N \log N + 4) \\ &= O(3N) + O(N \log N) + O(4) \\ &= O(N) + O(N \log N) + O(1) \\ &= O(N \log N) \end{aligned}$$

Exercise 3

Use the rules for Big-O to simplify these expressions:

(a) $O(6N^2 + 2N \log N + 3)$

Solution

$$O(N^2)$$

(b) $O(6 \times 2^N + 2N^2 \log N + 10^{1700} \times N)$

Solution

$$O(2^N)$$

Using the simplified algebra: expressions and assignments

For best and worst case analysis, don't count steps.

Use Big-O as an abstraction for counting.

Annotate each line of code with a $O(\cdot)$ expression.

Use the identities and simplifications.

Exercise 4

What's the Big-O for each line of code?

```
1 x = 3*y+4
2
3 y = data[2]*data[3]
4
5 z = 3*abs(data[i+1])
6
7 alist = [0]*N
```

Using the simplified algebra: single loop

Analyze the body of the loop using Big-O

Determine how often the loop is repeated.

Multiply.

Method 1: Step analysis

Example 1:

```
1 total = 0
2 for i in range(10):
3     total = total + 1
```

Line 1: 1 step

Just line 3, once: 2 steps

Just line 2, over-all: 10 assignments, 9 additions, so 19 steps

Line 3 is repeated 10 times, so 20 steps

Totals: $1 + 19 + 20 = 40$ steps

Asymptotically, $O(40) = O(1)$

Literally, the problem size here is **not a variable**.

Method 2: Simplified Steps

Example 1:

```
1 total = 0
2 for i in range(10):
3     total = total + 1
```

Line 1: 1 step, so $O(1)$

Just line 3, once: 2 steps, so $O(1)$

Just line 2, over-all: 10 assignments, 9 additions, 19 steps, so $O(1)$

Line 3 is repeated 10 times, so $10 \times O(1) = O(1)$

Totals: $O(1) + O(1) + O(1) = O(1 + 1 + 1) = O(1)$

Literally, the problem size here is **not a variable**.

Method 1: Step analysis

Example 2:

```
1 total = 0
2 for i in range(N):
3     total = total + 1
```

Line 1: 1 step

Just line 3, once: 2 steps

Just line 2, over-all: N assignments, $N - 1$ additions, so $2N - 1$ steps.

Line 3 is repeated N times, so $2N$ steps

Detailed step analysis: $1 + 2N - 1 + 2N = 4N$

Asymptotic category: $4N$ is $O(N)$.

Method 2: Simplified steps

Example 2:

```
1 total = 0
2 for i in range(N):
3     total = total + 1
```

Line 1: 1 step, so $O(1)$

Just line 3, once: 3 steps, so $O(1)$

Just line 2, over-all: N assignments, $N - 1$ additions, $2N - 1$ steps, so $O(N)$

Line 3 is repeated N times, so $N \times O(1) = O(N)$

Totals: $O(1) + O(N) + O(N) = O(2N + 1) = O(N)$

Exercises

Exercise 5

```
1 total = 0
2 for i in range(N):
3     total = total + 1
```

How many steps would Line 2 have to do so that the whole script is worse than $O(N)$?

Solution

If Line 2 is $O(N)$, the total cost is still $O(N)$. To make the whole thing worse, Line 2 would have to be worse than $O(N)$, say $O(N^2)$ or anything to the right of $O(N)$.

In other words, it doesn't matter if `range()` is $2N$ steps or $200N$ steps. Trying to count it exactly doesn't affect the asymptotic analysis.

Exercise 6

Analyze the following code, given each of the assumptions below:

```
1 total = 0
2 for i in range(N):
3     total = some_function(i, N)
```

(a) If `some_function(i, N)` were $O(1)$?

Solution

$O(N)$

(b) If `some_function(i, N)` were $O(\log N)$?

Solution

$O(N \log N)$

(c) If `some_function(i, N)` were $O(N^2)$?

Solution

$O(N^3)$

Variations in terminology

All of the following mean the same thing:

Analyze the program for its [asymptotic run time costs](#).

What is the [asymptotic time complexity](#) of the program?

Use [Big-O](#) to categorize the run time of the program.

Express the run time costs of the program using [Big-O notation](#).

Exercise 7

Express the run time costs of the following program using Big-O notation:

```
1 total = 0
2 i = 0
3 while i < N:
4     total = total + 1
5     i = i + 1
```

Solution

Lines 1,2,4,5 are all $O(1)$. Lines 4,5 are executed N times. The total cost is $O(N)$.

Logarithmic loops

```
1 total = 0
2 i = 1
3 while i < N:
4     total = total + 1
5     i = i * 2
```

Question: How many times does the loop-body execute?

Insight: The values for i are: 1, 2, 4, 8, ...

Answer: We can double i at most $\log_2 N$ times before $i > N$

Solution

Lines 1,2,4,5 are all $O(1)$. Lines 4,5 are executed $\log N$ times. The total cost is $O(\log N)$.

Logarithmic loops

```
1 total = 0
2 i = N
3 while i > 1:
4     total = total + 1
5     i = i // 2
```

Question: How many times does the loop-body execute?

Insight: The values for i are: $N, N/2, N/4, \dots$

Answer: We can half $i = N$ at most $\log_2 N$ times before $i \leq 1$.

Solution

Lines 1,2,4,5 are all $O(1)$. Lines 4,5 are executed $\log N$ times. The total cost is $O(\log N)$.

Using the simplified algebra: sequential loops

Analyze each loop separately

Add.

Exercise 8

```
1 total = 0
2 for i in range(N):
3     total = total + 1
4
5 for j in range(N):
6     total = total + 1
```

Solution

Lines 2-3 are $O(N)$. Lines 5-6 are $O(N)$. The total is $O(1) + O(N) + O(N) = O(N)$.

Nested loops with independent limits

Start with the inner-most loop

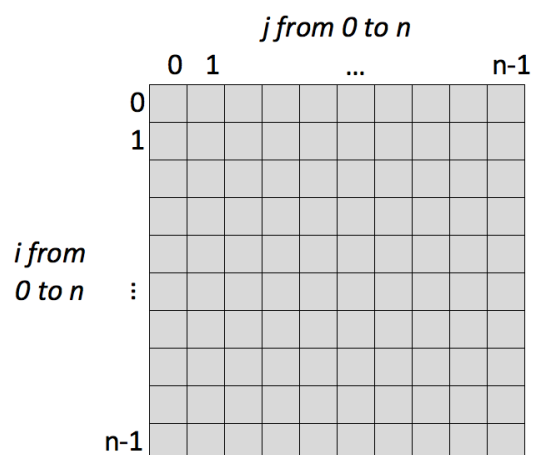
Analyze to find the number of steps

Abstraction: treat the inner-most loop as a statement with the analyzed cost.

Work outward, one loop at a time.

Exercise 9

```
1 total = 0
2 for i in range(N):
3     for j in range(N):
4         total = total + 1
```



Exercise 10

```
1 total = 0
2 for i in range(N):
3     for j in range(M):
4         total = total + 1
```

Assume M and N are separate input size parameters.

Solution

Lines 3-4 are $O(M)$. Repeating the inner loop N times gives a total of $N \times O(M) = O(NM)$.

Nested loops with dependent limits

```
1 total = 0
2 for i in range(N):
3     for j in range(i):
4         total = total + 1
```

When $i == 0$, lines 3-4 repeated 0 times

When $i == 1$, lines 3-4 repeated 1 times

When $i == 2$, lines 3-4 repeated 2 times

When $i == N-1$, lines 3-4 repeated $N-1$ times

In general, lines 3-4 are repeated $O(i)$ times

Nested loops with dependent limits

```
1 total = 0
2 for i in range(N):
3     for j in range(i):
4         total = total + 1
```

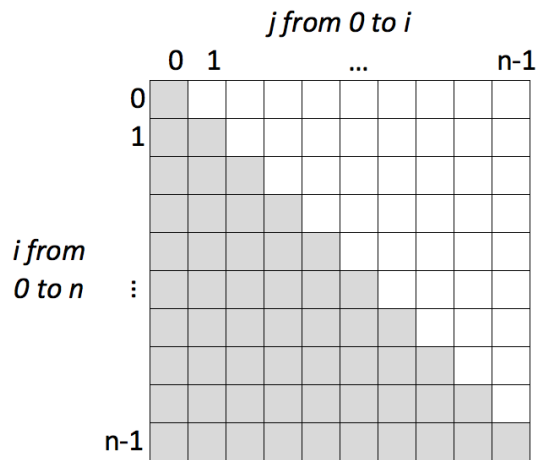
In general, lines 3-4 are repeated $O(i)$ times

Total cost: sum the costs for i from 0 to $N - 1$

Total: $O(0) + O(1) + O(2) + \dots + O(i) + \dots + O(N - 1)$

An identity you must memorize

$$\sum_{i=1}^n i \equiv 1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$$



Nested loops with dependent limits

```

1 total = 0
2 for i in range(N):
3     for j in range(i):
4         total = total + 1

```

In general, lines 3-4 are repeated $O(i)$ times

Total cost: sum the costs for i from 0 to $N - 1$

Total: $O(0) + O(1) + O(2) + \dots + O(N - 1) = O(N^2)$

Nested loops with dependent limits

Start with the inner-most loop

Analyze to find the number of steps expressed using dependency

Abstraction: treat the inner-most loop as a statement with the analyzed cost.

Sum the total costs of the dependent loop

Work outward, one loop at a time.

If statements

Analyze the cost of the condition(s)

Analyze the cost of each branch

Worst case analysis: Use the cost of **the most expensive branch** only

Best case analysis: Use the cost of **the least expensive branch** only

Exercise 11

What is the asymptotic time complexity of the following program?

```

1 sign = 0
2 if N < 0:
3     sign = -1
4 else:
5     sign = 1

```

Solution

Both branches are $O(1)$. Everything else is $O(1)$.

Exercise 12

Analyze the following program for its asymptotic run time costs.

```
1 total = 0
2 i = 0
3 while i < N:
4     if i % 2 == 0:
5         total = total + 1
6     i = i + 1
```

Solution

The branch is $O(1)$. It's done half the time. The body of the loop is $O(1)$. The body is done N times. Total: $O(N)$.

Exercise 13

Analyze the following program for its asymptotic run time costs.

```
1 total = 0
2 i = 0
3 while i < N:
4     if i < N//2:
5         total = total + i
6     else:
7         total = total + 2*i
8     i = i + 1
```

Solution

The two branches have slightly different costs, by 1 step, but both are $O(1)$. Best case and worst case is the same asymptotically, so the total cost is $O(N)$.

Exercise 14

Analyze the following program for its asymptotic run time costs.

```
1 total = 0
2 i = 0
3 while i < N:
4     if i % 2 == 0:
5         for j in range(N):
6             total = total + j
7     else:
8         total = total + 2*i
9     i = i + 1
```

Solution

Here the one branch is $O(N)$, but the other is $O(1)$.

If you're not careful, you might say worst case for the body of the loop (Lines 4-9) is $O(N)$, and the best case is $O(1)$.

However, the two branches alternate, so both branches are executed half the time. So we can't say the best case is better than the worst case. For any N , the cost of the body of the loop is about the same.

The total cost is N times the cost of the body of the loop, so $O(N^2)$.

Exercise 15

What is the asymptotic time complexity of the following program?

```
1 total = 0
2 for value in list_of_numbers:
3     if value % 2 == 0:
4         total = total + 1
```

Solution

Here, the best case is if all the numbers in the list are odd. The worst case is if all the numbers are even. The difference is just a couple of steps, so, asymptotically, no difference at all.

The total cost is $O(N)$, where N is the length of the list.

Exercise 16

What is the asymptotic time complexity of the following program?

```
1 total = 0
2 for value in list_of_numbers:
3     if value < k:
4         total = total + value
5     else:
6         total = total + 2*value
```

Solution

Another example of a negligible difference between best and worst cases.

The total cost is $O(N)$, where N is the length of the list.

Exercise 17

```
1 total = 0
2 for value in list_of_numbers:
3     if value % 2 == 0:
4         total = total + 1
5     else:
6         for i in range(len(list_of_numbers)):
7             total = total + 1
```

Solution

Here, best and worst cases are meaningful, because we can't predict what will happen. We could have a list of all even numbers, which would be the best case. In the best case, the cost of the body of the loop (lines 3-7) is $O(1)$. We could have a list of all odd numbers, which would be the worst case. In the worst case, the cost of the body of the loop is $O(N)$.

The total worst case cost: $O(N^2)$.

The total best case cost: $O(N)$.

The average case would require us to have some idea how often even and odd numbers appear in the list. That could be impossible to predict, unless we knew something about a larger application using this code. That's why average case analysis is hard. Also, if you "guess" that the expensive branch will happen about half the time, and use that as your "guess" about averages, then the average case is $0.5 \times O(N) + 0.5 \times O(N^2) = O(N^2)$. No better than the worst case analysis.

Exercise 18

Categorize the following program in terms of Big-O, by counting steps. What is the Big-O analysis for the best and worst case?

```
1 def roll_dice():
2     return random.randint(1,6) + random.randint(1,6)
3
4 def rolling(n_rolls):
5     low_rolls = 0      # number rolls whose sum is <= 4
6     high_rolls = 0     # number rolls whose sum is >= 10
7     dice_rolls = [0]*n_rolls
8     for roll in range(n_rolls):
9         dice_rolls[roll] = roll_dice()
10        if dice_rolls[roll] <= 4:
11            low_rolls += 1
12        elif dice_rolls[roll] >= 10:
13            high_rolls += 1
14    print(low_rolls, high_rolls)
15    print(dice_rolls)
```

Solution

The size parameter is `n_rolls`, which I will call N .

Lines 5-6 are $O(1)$. Line 7 is $O(N)$, because it is constructing a list.

The function call on Line 9 is $O(1)$, assuming that `randint()` is $O(1)$, which is a reasonable assumption.

Lines 10-13 are all $O(1)$.

The total cost of the body of the loop (Lines 9-13) is $O(1)$, and the body is repeated N times.

The print statement on line 14 is $O(1)$, and the one on line 15 is $O(N)$, because it's displaying a list with length N .

In total, $O(N)$.

Exercise 19

```
1 # n_players is a pre-defined integer
2 round_robin = np.zeros([n_players, n_players], dtype=int)
3 games_per_match = 5
4 for p1 in range(n_players):
5     for p2 in range(p1+1, n_players):
```

```
6     p1_games_won = random.randint(1,games_per_match)
7     round_robin[p1,p2] = p1_games_won
8     round_robin[p2,p1] = games_per_match - p1_games_won
9 print(round_robin)
```